TCP-Fusion: A Hybrid Congestion Control Algorithm

Implementation and Performance Evaluation in NS-3

Aritra Debnath

Outline

1. Problem Statement
2. TCP-Fusion Solution
3. Algorithm Details
4. Expected Performance
5. Implementation Plan
6. Challenges & Conclusion

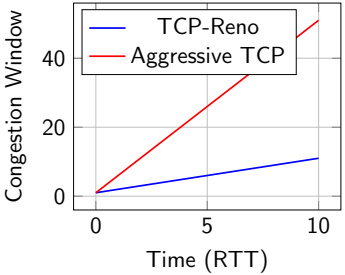
The Challenge in High-Speed Networks

Current State of the Internet:

- Billions of devices use TCP-Reno
- High-speed networks (1 Gbps+) are common
- Cannot replace all implementations simultaneously

The Problems:

1. TCP-Reno: Too slow for high-speed networks
2. New variants (HSTCP, BIC): Unfair to TCP-Reno
3. Deployment barrier: Must coexist fairly



TCP-Reno increases too slowly

TCP-Reno Limitations

Two-Phase Operation

Slow Start (Exponential):

- Fast growth: $\text{cwnd} = 2^{\text{RTT}}$
- Stops at *ssthresh* or packet loss
- Works well for small transfers

Congestion Avoidance (Linear):

- $\text{cwnd} = \text{cwnd} + 1$ per RTT
- Very slow in high-speed networks!
- Example: 1 Gbps, 100ms RTT $\rightarrow$ **20 minutes** to fill pipe!

The Dilemma

Cannot simply increase *ssthresh* $\rightarrow$ Causes unfairness and packet losses!

TCP-Fusion: Hybrid Approach

Core Concept

Combines **three proven approaches** into one adaptive algorithm:

TCP-Reno
(Loss-based)

AIMD
Fairness
Baseline

TCP-Vegas
(Delay-based)

RTT-based
Early
Detection

TCP-Westwood
(Bandwidth Est.)

Smart
Window
Reduction

TCP-Fusion

Efficiency + Fairness + Robustness

Key Innovation: Dual Window Mechanism

TCP-Fusion Maintains Two Windows Simultaneously

$$\boxed{\text{actual_cwnd} = \max(\text{fusion_cwnd}, \text{reno_cwnd})}$$

Fusion Window:

- Smart, adaptive behavior
- Uses RTT measurements
- Aggressive when safe
- Conservative when competing

Reno Window:

- Standard AIMD
- Always running in parallel
- Safety guarantee
- Ensures fairness baseline

Result

At least TCP-Reno performance + Higher throughput when possible

TCP-Fusion Algorithm: Core Logic

Step 1: Measure Network State

$$\text{diff} = \text{cwnd} \times \frac{\text{RTT} - \text{RTT}_{\text{min}}}{\text{RTT}}$$

$$\alpha = \frac{\text{cwnd} \times D_{\text{min}}}{\text{RTT}} \quad \text{(Adaptive threshold)}$$

where $D_{\text{min}} = 4\text{ms}$ (queuing delay tolerance)

Step 2: Determine Operating Mode

- **Underutilized** ($\text{diff} < \alpha$): Network has unused capacity
- **Balanced** ($\alpha \leq \text{diff} \leq 3\alpha$): Network fully utilized / competing
- **Congested** ($\text{diff} > 3\alpha$): Early congestion detected

Window Update Rules

Algorithm 1 TCP-Fusion Congestion Window Update

Calculate $\text{diff} = \text{cwnd} \times \frac{\text{RTT} - \text{RTT}_{\text{min}}}{\text{RTT}}$
Calculate $\alpha = \frac{\text{cwnd} \times 4\text{ms}}{\text{RTT}}$
if $\text{diff} < \alpha$ **then**
    $\text{fusion_cwnd} \leftarrow \text{cwnd} + W_{\text{inc}}$ {Aggressive increase}
else if $\text{diff} > 3\alpha$ **then**
    $\text{fusion_cwnd} \leftarrow \text{cwnd} + \frac{\alpha - \text{diff}}{\text{cwnd}}$ {Gentle decrease}
else
    $\text{fusion_cwnd} \leftarrow \text{reno_cwnd}$ {Match Reno (fairness)}
end if
$\text{reno_cwnd} \leftarrow \text{reno_cwnd} + \frac{1}{\text{cwnd}}$ {Always update Reno window}
$\text{actual_cwnd} \leftarrow \max(\text{fusion_cwnd}, \text{reno_cwnd})$

$W_{\text{inc}} = \frac{B \times 1\text{ms}}{12\text{Mbps}}$ where B = estimated bandwidth

Window Reduction on Packet Loss

Smart Reduction (from TCP-Westwood)

$$\text{cwnd}_{\text{new}} = \max\left(\text{cwnd} \times \frac{\text{RTT}_{\text{min}}}{\text{RTT}}, \frac{\text{cwnd}}{2}\right)$$

Traditional TCP-Reno:

- Always cwnd $\leftarrow \frac{\text{cwnd}}{2}$
- Drastic reduction
- Wastes bandwidth
- Poor for random losses

TCP-Fusion:

- Adaptive reduction
- Based on buffer occupancy
- Gentler for random losses
- Better for wireless links

When $\text{RTT} = 2 \times \text{RTT}_{\text{min}}$ (buffer full)

Reduction factor $= \frac{1}{2}$ (same as Reno) $\rightarrow$ **Fair!**

Adaptive Parameters

Why Dynamic Thresholds?

Unlike fixed *ssthresh*, TCP-Fusion adapts to:

- Link bandwidth (scales with capacity)
- Number of competing flows (fair sharing)
- Network conditions (RTT variations)

Parameter α:

$$\alpha = \frac{\text{cwnd} \times 4\text{ms}}{\text{RTT}}$$

- Increases with bandwidth
- Decreases with more flows
- Self-regulating

Parameter W_{inc}:

$$W_{\text{inc}} = \frac{B \times 1\text{ms}}{12\text{Mbps}}$$

- Based on bandwidth estimate
- Prevents buffer overflow
- Scales to network speed

Performance Improvements: Efficiency

Metric	TCP-Reno	TCP-Fusion
Throughput (1 Gbps link)	400 Mbps	**950+ Mbps**
Link utilization	40-50%	**95-99%**
Time to fill pipe (100ms RTT)	20+ minutes	**2-3 seconds**
Performance in lossy links (10^{-4})	Baseline	**1.4× better**

Table: Expected efficiency improvements over TCP-Reno

Key Achievement

30-40% throughput improvement in high-speed networks

Performance Improvements: Fairness

Scenario	Expected Fairness Index	Behavior
Fusion vs. Reno (saturated link)	$\approx$ **1.0**	50-50 share
Fusion vs. Reno (unused capacity)	$>$ **1.0**	Fusion uses extra
Multiple Fusion flows	$\approx$ **1.0**	Fair convergence
RTT fairness	**Similar to Reno**	No severe bias

Table: Expected fairness characteristics

Jain's Fairness Index

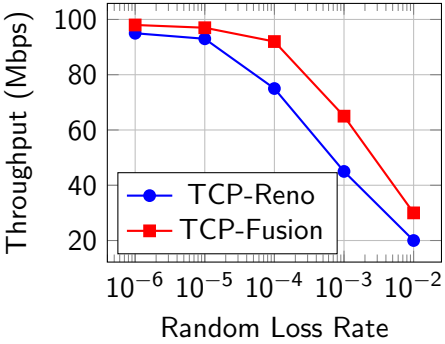
$$\text{Fairness} = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \sum_{i=1}^{n} x_i^2}$$

where x_i is throughput of flow i. Perfect fairness $= 1.0$

Performance Improvements: Robustness

Handling Various Conditions:

- **Random packet losses:** Smart reduction handles wireless/lossy links better
- **High BDP networks:** Adaptive parameters scale automatically
- **Variable conditions:** RTT-based detection adapts in real-time
- **Mixed traffic:** Fair to both Fusion and Reno flows



Better performance in lossy environments

Comparison with Other TCP Variants

TCP Variant	Speed	Fairness	Robustness	Deployable?
TCP-Reno	Poor	Excellent	Poor	✓ Yes
HSTCP	Excellent	Poor	Medium	× No
BIC	Excellent	Poor	Medium	Limited
CUBIC	Very Good	Poor	Good	Partial
TCP-Vegas	Good	Poor*	Good	× No
TCP-Westwood	Good	Medium	Excellent	Partial
TCP-Fusion	**Excellent**	**Excellent**	**Excellent**	✓ **Yes**

Table: Qualitative comparison of TCP variants

*Vegas is fair to other Vegas flows but unfair to Reno

TCP-Fusion's Advantage

Only variant achieving **high speed + fairness + deployability**

Implementation Scope

What Will Be Implemented in NS-3

1. **RTT Tracking Module**
   - Track minimum RTT and current RTT
   - Calculate diff and adaptive α
2. **Bandwidth Estimation**
   - Monitor ACK arrival rate
   - Estimate available bandwidth for W_{inc}
3. **Dual Window Management**
   - Maintain both fusion_cwnd and reno_cwnd
   - Selection logic: max(fusion_cwnd, reno_cwnd)
4. **Congestion Control Logic**
   - Window increase (3 modes)
   - Smart window reduction on loss

Implementation in C++ using NS-3 framework

Testing Scenarios

Test 1: Single Flow Performance

- Vary: Link speed (100M-1G)
- Vary: RTT (20-200ms)
- Vary: Loss rate (10^{-6} to 10^{-3})
- Measure: Throughput, utilization

Test 2: Fairness with TCP-Reno

- 1 Fusion + 1 Reno flow
- Vary: Buffer size, capacity
- Measure: Fairness index

Test 3: Multiple Fusion Flows

- 2-5 competing flows
- Measure: Convergence time

Test 4: Lossy Networks

- Wireless/high-loss scenarios
- Loss rate: 10^{-4} to 10^{-3}
- Compare: Fusion vs Reno vs Westwood

Test 5: Mixed RTT

- Flows with different RTTs
- RTT: 20ms, 80ms, 160ms
- Measure: RTT fairness

Baseline Comparisons:

- TCP-Reno (NS-3 default)
- TCP-Vegas (NS-3)
- TCP-Westwood (NS-3)

Project Timeline

Phase	Tasks
Study	Read TCP-Fusion paper, understand algorithm details, study NS-3 TCP i
Core Implementation	Implement RTT tracking, diff calculation, adaptive α mechanism
Window Logic	Implement dual window management, fusion_cwnd update logic (3 modes
Completion	Implement window reduction, bandwidth estimation, integrate all compon
Testing	Debug, initial simulation runs, verify basic functionality
Evaluation	Run all 5 test scenarios, collect data, analyze results
Documentation	Create graphs, write report, prepare presentation

Expected Deliverables

1. **Implementation:**
   - TCP-Fusion module in NS-3
   - Simulation scripts for all tests
   - Well-commented code
   - Configuration files
2. **Performance Analysis:**
   - Throughput comparison graphs
   - Fairness index calculations
   - Convergence time plots
   - Statistical analysis (mean, std dev)
3. **Documentation:**
   - Algorithm flowchart
   - Parameter tuning guide
   - Implementation challenges
   - Future improvements
4. **Presentation:**
   - Results visualization
   - Demo of simulations
   - Performance comparison tables
   - Lessons learned

Success Criteria

Demonstrate **improved efficiency** while maintaining **fairness to TCP-Reno**

Disadvantages of TCP-Fusion

Implementation Complexity

- More complex than traditional TCP variants
- Requires maintaining dual window state
- Multiple parameters to tune (α, W_{inc}, D_{min})
- Higher computational overhead for RTT tracking and bandwidth estimation

Deployment Challenges

- Not standardized - requires custom implementation
- Limited real-world testing compared to established variants
- May require kernel modifications for deployment
- Interactions with modern features (ECN, SACK) need validation

Performance Trade-offs

- RTT measurement accuracy critical - noisy measurements can degrade performance

Potential Challenges & Solutions

Challenge	Issue	Solution
RTT Measurement	Accurate min/current RTT needed	Use NS-3's existing RTT estimator
Parameter Tuning	Choosing optimal $D_{\min}$, β	Start with paper values (4ms, 12Mbps), experiment
Bandwidth Estimation	Complex implementation	Use simplified ACK-rate based estimation
NS-3 Integration	Modifying TCP stack	Extend TcpSocketBase or TcpCongestionOps class
Testing Complexity	Many scenarios to validate	Use NS-3's TCP test framework as template

Risk Mitigation

Why TCP-Fusion?

Academic Value:

- Published, peer-reviewed algorithm
- Strong theoretical foundation
- Real-world applicability
- Multi-concept integration

Learning Outcomes:

- Deep understanding of TCP
- Delay vs. loss-based control
- Fairness vs. efficiency tradeoffs
- NS-3 protocol implementation

Project Benefits:

- Not directly in NS-3 (original work)
- Manageable scope (5-7 days)
- Clear improvement story
- Strong evaluation potential

Practical Relevance:

- Addresses real problem
- Deployable solution
- High-speed networks focus
- Modern network challenges

Conclusion

Project Summary

TCP-Fusion combines loss-based, delay-based, and bandwidth estimation techniques into a hybrid congestion control algorithm that achieves:

Efficiency	Fairness	Robustness
30-40% better throughput	Fair to TCP-Reno	Handles losses well
95-99% link utilization	Deployable solution	Works in various scenarios
Fast convergence	Adaptive behavior	Self-regulating

Expected Outcome

A working NS-3 implementation demonstrating that **high performance and fairness can coexist**

References

- Kaneko, K., Fujikawa, T., Su, Z., & Katto, J. (2006). *TCP-Fusion: A Hybrid Congestion Control Algorithm for High-speed Networks*. Graduate School of Science and Engineering, Waseda University.
- Stevens, W. R. (1997). *TCP Slow Start, Congestion Avoidance, Fast Retransmit, and Fast Recovery Algorithms*. IETF RFC 2581.
- Brakmo, L. S., & Peterson, L. L. (1995). *TCP Vegas: End-to-End Congestion Avoidance on a Global Internet*. IEEE Journal on Selected Areas in Communication, Vol. 13.
- Casetti, C., Gerla, M., Mascolo, S., Sanadidi, M. Y., & Wang, R. (2001). *TCP Westwood: Bandwidth Estimation for Enhanced Transport over Wireless Links*. ACM Mobicom 2001.
- NS-3 Network Simulator. `https://www.nsnam.org/`

Thank You!

Paper: `http://www.csc.lsu.edu/~sjpark/cs7601/1-TCP_fusion.pdf`